

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Student Name: B.Charishma

Roll No.: a24126552067

Date: 30-08-2026

1. TITLE

API Development and Advanced Routing using the Express.js Framework

2. AIM

To architect a scalable backend service using the Express.js framework, implementing basic HTTP routing to serve both static HTML views and structured JSON Application Programming Interfaces (APIs).

3. OBJECTIVE

The objectives of this experiment are:

- To install and initialize the Express.js framework within a Node application.
- To configure middleware and port bindings.
- To construct GET route handlers for the root URL endpoint.
- To design a dedicated API endpoint serving complex JavaScript objects.
- To automate the serialization of JavaScript objects into JSON format for network transport.

4. THEORY

Express.js is a fast, unopinionated, minimalist web framework for Node.js. While Node's native HTTP module is powerful, creating complex routing logic is cumbersome. Express abstracts this complexity, providing a robust set of features for web and mobile applications.

Routes determine how the application responds to client requests at particular endpoints, which consist of a URI (or path) and a specific HTTP request method (GET, POST, PUT, DELETE).

Application Flow:

Express App Instance -> Define Endpoint Definitions (/ and /profile) -> Bind Route Handlers -> Listen on Port 3000 -> Automatic JSON Serialization on Request

5. ALGORITHM / PROCEDURE

1. Import the express module using CommonJS require.
2. Initialize the application instance by executing `express()`.
3. Define an arbitrary port variable (e.g., 3000).
4. Define a root route (`/`) using `app.get()` and utilize `res.send()` to return an HTML string.
5. Define a secondary API route (`/profile`) targeting data retrieval.
6. Create a local JavaScript object containing structured profile details (Name, Dept, CGPA).
7. Utilize the `res.json()` method to automatically serialize and dispatch the object.
8. Call `app.listen()` to bind the application to the network port.
9. Verify functionality by accessing both endpoints via a web browser.

6. PROGRAM / SOURCE CODE

Source File: week6.js

```

// Import the Express module
const express = require('express');

// Create the application
const app = express();
const port = 3000;

// Route 1: The Home Page s
app.get('/', (req, res) => {
  res.send(`
    <h1>Welcome to my Express Server</h1>
    <p>This is the home page. Try visiting <b>localhost:3000/profile</b></p>
  `);
});

// Route 2: A Data API
app.get('/profile', (req, res) => {

  // A simple Javascript object holding student details
  const myProfile = {
    name: "B.Charishma",
    department: "CSE (AI & ML)",
    cgpa: 8.35,
    status: "Active"
  };

  // Express automatically converts the JS object to JSON and sends it!
  res.json(myProfile);
});

// Start the server and listen for requests
app.listen(port, () => {
  console.log(`Express app running at http://localhost:${port}`);
});

```

7. CODE EXPLANATION

Code / Syntax	Explanation
<code>const app = express();</code>	Creates an Express application object, providing routing and middleware capabilities.
<code>app.get('/route', callback)</code>	Routes HTTP GET requests to the specified path with the specified callback functions.
<code>res.send('HTML')</code>	Sends the HTTP response, automatically assigning the text/html content-type header.

<code>res.json(object)</code>	Sends a JSON response, automatically converting the JS object via <code>JSON.stringify()</code> .
<code>app.listen(port, callback)</code>	Binds and listens for connections on the specified host and port.

8. TEST CASES AND EXECUTION DATA

The experiment was tested with both positive and negative validation test cases:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Root Endpoint Routing	Send GET request to localhost:3000/	Express intercepts request and responds with HTML Welcome page	Pass
TC-02	API Endpoint Routing	Send GET request to localhost:3000/profile	Express executes specific handler, returning Profile object	Pass
TC-03	404 Not Found Handling	Access undefined endpoint /invalid	Express falls back to default 404 handler, returning 'Cannot GET /invalid'	Fail

9. OUTPUT

Output 1: /profile API Endpoint JSON Response

```
{"name": "B.Charishma", "roll": "a24126552067", "dept": "CSE"}
```

Output 2: Root Route HTML Rendering

Welcome to Charishma Express Server!

10. TEST CASES SCREENSHOTS

Test Case 1: Route Invalid Check

Cannot GET /invalid

11. RESULT

Thus, the experiment was successfully implemented and verified across both positive functional and negative validation test cases.